



How to use Unsloth as an API endpoint

You can run **local LLMs** with tools like [Claude Code](#) and [Codex](#) by connecting those tools to Unsloth's **OpenAI-compatible API endpoint**. This lets you run models like [Qwen](#) and [Gemma](#) locally for agentic coding. Unsloth also has beneficial features such as self-healing **tool calling**, **code execution**, and **web search**.

Unsloth makes it easy to deploy a fast API inference endpoint that provides:

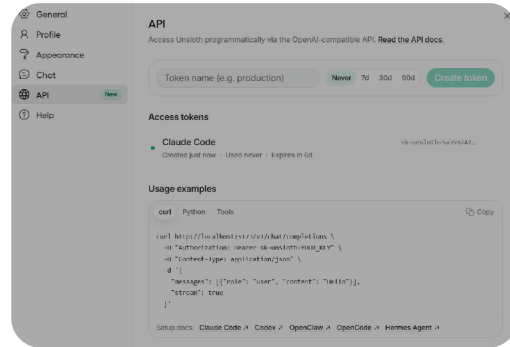
- [Self-healing tool calling](#), which helps reduce broken or malformed tool calls by 50%
- [Code execution](#) support, allowing Bash and Python execution for more accurate code outputs.
- **Advanced [Web search](#)** that visits and actually reads webpages to gather in-c
- [Automatic in](#) etc.)

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

Accept

Reject

Models loaded in Unsloth (including GGUFs) are exposed as an **authenticated API** via `llama-server`. A long API key is generated for security reasons like how OpenAI provides one.



Your **local models** can then be used directly in your preferred AI agent, SDK, or chat client. Unsloth speaks two dialects on the same port. Both support streaming, tool calling (OpenAI `tools` / Anthropic `tools`), and vision inputs:

- **Anthropic-compatible** `/v1/messages` for Claude Code, OpenClaw, the Anthropic SDK, and any client that expects the Messages API.
- **OpenAI-compatible** `/v1/chat/completions` and `/v1/responses` for the OpenAI SDK, OpenCode, Cursor, Continue, Cline, Open WebUI, SillyTavern, and any OpenAI-compatible tool.

⚡ Quickstart

1. **Install or update Unsloth Studio.** Then launch Unsloth

2. **Load a model** and wait for it

3. **Create an API key** → **Settings**

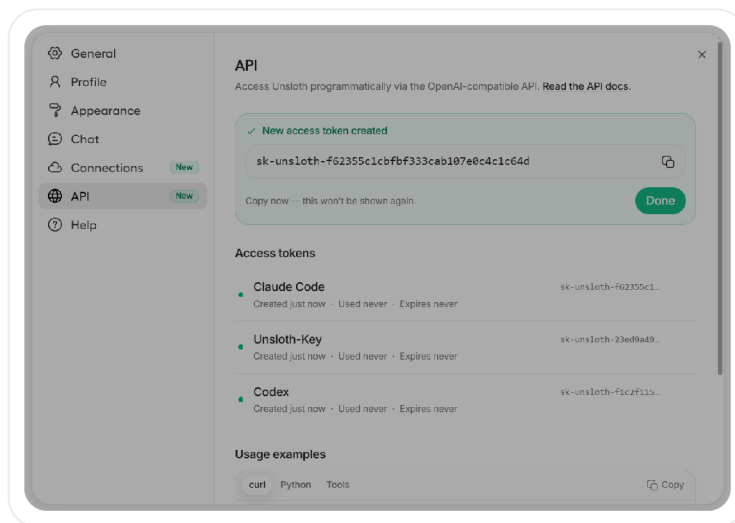
`unsloth-...` value that appears. Unsloth only shows it once.

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

4. **Point your client at Unsloth.** Use `http://localhost:PORT` as the base URL and your `sk-unsloth-...` key for auth. Jump to the recipe for your tool below.

🔑 Creating an API key

1. Open the sidebar, click your **Unsloth** avatar at the bottom-left.
2. Go to **Settings** → **API** (globe 🌐 icon).
3. Enter a friendly name (e.g. `claude-code-macbook`). Set an expiry (optional)
4. Click **Create**.
5. **Copy the key.** Unsloth stores only a hash and you won't be able to view it again.



All keys start with the `sk-unsloth-` prefix. Revoke a key from the same page at any time. If you receive a `401 Unauthorized` error, it means your key is invalid.

⚠️ Treat your API key like a password. Only share it with trusted parties. It gives you access to your model.

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

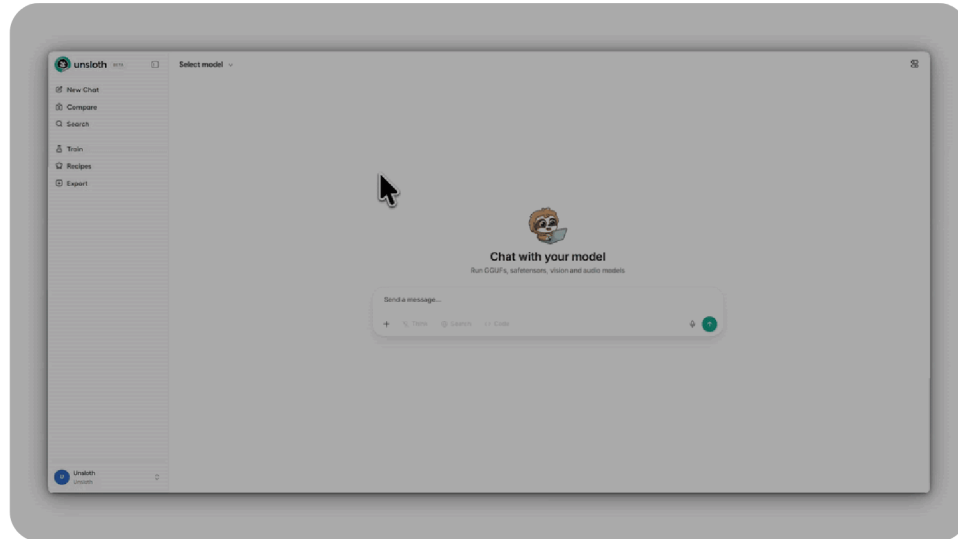


Model Loading

1

Select Model

Before using the API, load a model from the **Select model** dropdown in the top-left corner of the Chat page.



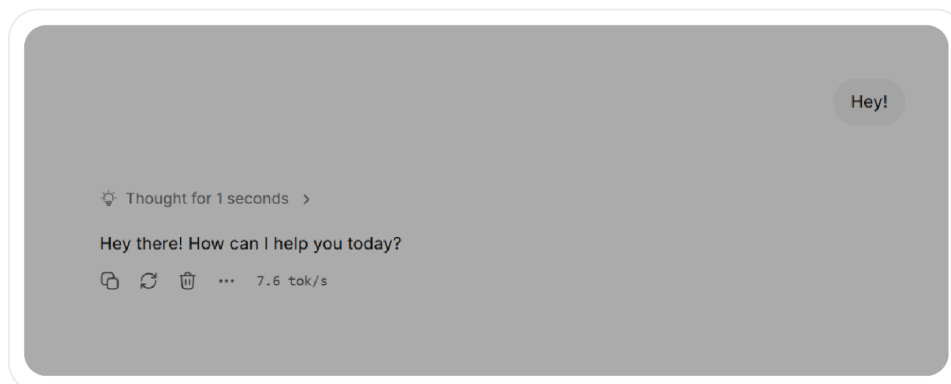
In this guide, we'll use:

`unsloth/gemma-4-26B-A4B-it-GGUF` with the recommended `UD-Q4_K_XL` quantization.

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

2 Test the Model

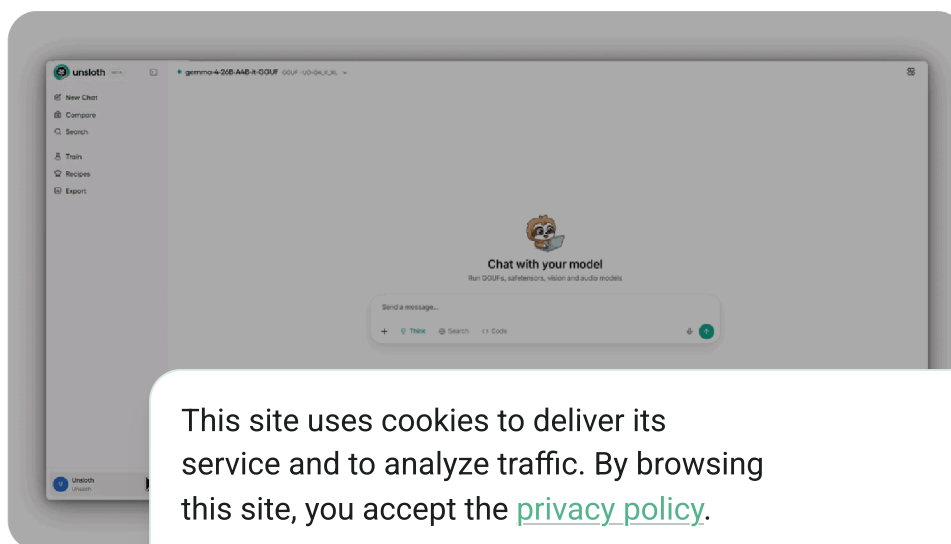
Before using the Client, send a quick message:



 This confirms that the model loaded correctly and is ready to respond.

3 Unsloth API key

In Unsloth, open **Settings** → **API** to view or create your API key.



Treat your
screenshots or repositories.

>_ Unsloth run command

1. **Install or update Unsloth Studio.** Earlier versions don't expose the external API. See Installation.
2. **Load a GGUF model.** load a GGUF model using the run command. This will also load the UI on the default port. The endpoint URL and API Key will be printed out to the console , ready for you to be used with your client of choice.

```
unsloth run --model unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL
```

Loading a model from the CLI

You can load a model and have an API key created for you automatically using the `unsloth` CLI tool. When the model finishes loading, the endpoint URL and API key are printed to your console. Copy them into your client of choice and you're ready to go.

Before you start

Make sure you're on a recent version of Unsloth Studio as earlier versions don't expose the external API. See [installation](#).

The quick way

Open a terminal

```
unsloth run --mo
```

This starts the s
your endpoint URL and API key.

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

How the model name works

You can point at a model in a few different ways. Pick the one you find easiest:

```
# Combined: repo and quantization variant in one string (recommended
— shortest)
```

```
unsloth run --model unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL
```

```
# Separate: repo and variant as two flags (the older style, still works)
```

```
unsloth run --model unsloth/gemma-4-26B-A4B-it-GGUF --gguf-variant
UD-Q4_K_XL
```

```
# Using -hf / --hf-repo (matches llama.cpp's spelling, handy if you're
coming from there)
```

```
unsloth run -hf unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL
```

Tuning the run (optional)

You don't need any of this for a basic load, but `unsloth run` supports many llama-server runtime flags for customizing performance, memory usage, context length, generation behavior, networking, and tool access.

Additional flags are forwarded directly to the underlying inference server, and your values override Unsloth's defaults.

Adjust generation

Sampling settings
the model behavior

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

```
# Lower randomness and improve reproducibility
unsloth run \
  --model unsloth/Qwen3-1.7B-GGUF \
  --temp 0.6 \
  --seed 42
```

Lower temperature values usually produce more stable outputs, while top-p, top-k, min-p, and repeat penalty settings further control token selection and repetition.

```
# Tune token selection and repetition behavior
unsloth run \
  --model unsloth/Qwen3-1.7B-GGUF \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.05 \
  --repeat-penalty 1.1
```

Increase context length and CPU threads

Useful if you're working with large projects, long chats, or agent workflows that need more memory.

```
# Use a larger context window and more CPU threads
unsloth run \
  --model unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL \
  -c 131072 \
  --threads 32
```

Expose the API

By default, Unsloth does not expose the API to the world.

0.0.0.0

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).


```
# Allow LAN devices to connect
unsloth run \
  --model unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL \
  -H 0.0.0.0 \
  -p 8888
```

Control reasoning behavior

Some reasoning-capable models support additional flags for controlling thinking and reasoning behavior.

```
# Disable reasoning / thinking output
unsloth run \
  --model unsloth/Qwen3-1.7B-GGUF \
  --reasoning off
```

```
# Enable reasoning mode
unsloth run \
  --model unsloth/Qwen3-1.7B-GGUF \
  --reasoning on
```

Reasoning support depends on the model and backend capabilities.

Enable or disable server-side tools

Control whether tools like web search and code execution are exposed by the i

```
# Explicitly enable tools
unsloth run \
  --model unsloth/...
  --enable-tools
```

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

```
# Explicitly disable tools
unsloth run \
  --model unsloth/gemma-4-26B-A4B-it-GGUF:UD-Q4_K_XL \
  --disable-tools
```

Unsloth supports most llama-server runtime flags, including context sizing, GPU layers, threading, sampling, networking, and tool configuration.

See the [llama-server](#) documentation for the full list of supported runtime flags.

Server-side tool policy

`unsloth run` controls whether server-side tools (web search, code execution, etc.) are exposed by the inference server. Defaults are based on the bind address:

- `127.0.0.1` (**localhost**) — tools **on** by default. Only your machine can reach the server.
- `0.0.0.0` or any **non-loopback address** — tools **off** by default. A leaked API key on a network-exposed server means arbitrary code execution on the host.

Flags:

- `--enable-tools` / `--disable-tools` — force on or off. On `0.0.0.0`, `--enable-tools` s
- `--yes` / `-y` —

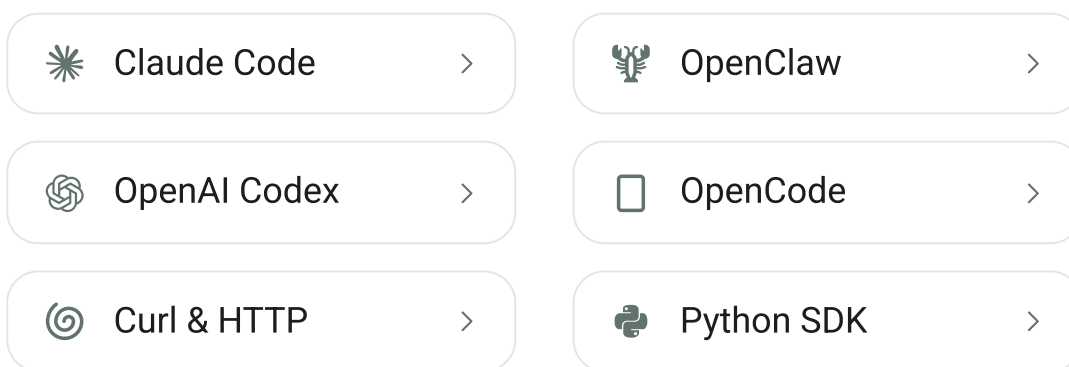
The resolved po
requests cannot

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

 You don't need to run different servers for the two formats. Unsloth handles both on the same port.

Connecting your client

Unsloth enables you run local LLMs via most frameworks including [Claude Code](#), [Codex](#), [OpenClaw](#), [OpenCode](#) and more. Click the specific tools below for a guide:



Tool calling

Both endpoints support function / tool calling in their native format, plus an Unsloth-specific shorthand for Unsloth's built-in tools.

OpenAI-style tools: send `tools` and `tool_choice` to `/v1/chat/completions` exactly as you would with OpenAI. Claude Code (via `/v1/messages`), `opencode`, `Cursor`, `Continue`, and `Cline` all work out of the box.

Anthropic-style

`tool_choice` to `/`

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

Unsloth server side tools: Unsloth can execute Python, web search, and bash *server-side* and stream the results back as `tool_result` events. Opt in by adding these extra fields to either endpoint:

```
{
  "messages": [{"role": "user", "content": "What is 123 * 456? Use Python."}],
  "stream": true,
  "enable_tools": true,
  "enabled_tools": ["python", "web_search", "terminal"],
  "session_id": "my-session"
}
```

The model sees each tool's output on its next turn. For deeper coverage (schemas, streaming events, chaining), see .

❗ If you're using the Anthropic `/v1/messages` endpoint, `tool_choice` maps cleanly: Anthropic `auto` → OpenAI `auto`, Anthropic `any` → OpenAI `required`, Anthropic `{type: "tool", name: "x"}` → OpenAI `{type: "function", function: {name: "x"}}`, Anthropic `none` → OpenAI `none`.

? Troubleshooting

401 Unauthorized : either the `Authorization` header is missing or the key is wrong. Keys must be passed as `Authorization: Bearer sk-unsloth-...`. If you lost the key, create a new one from **Settings** → **API**. Unsloth doesn't show old keys after creation.

Lost connection t
underlying llama
crashed, or the r
model from **New**

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

Claude Code shows the default Anthropic model, not my local one

: check all three env vars are exported in the **same** shell where you run `claude`:

```
echo $ANTHROPIC_BASE_URL
echo $ANTHROPIC_AUTH_TOKEN
echo $ANTHROPIC_MODEL
```

Then run `/model` inside Claude Code to confirm. On Windows PowerShell use `$env:ANTHROPIC_BASE_URL` etc.

stream: true returns a single JSON blob instead of SSE : make sure you're hitting the right path (`/v1/messages` or `/v1/chat/completions`) and that your HTTP client is actually consuming the response as a stream, not buffering it.

I can't find the name of the model to add to opencode (or OpenClaw / any other client) : ask Unsloth directly. `GET /v1/models` returns the exact model ID you need to plug into the client's "Model ID" field:

```
curl http://localhost:8888/v1/models \
-H "Authorization: Bearer sk-unsloth-xxxxxxxxxxxx"
```

You'll get back a JSON payload of the form `{"data": [{"id": "gemma-4-26B-A4B-it-GGUF", ...}]}`. Copy the `id` value, that's the string opencode's **Model ID** field (left column) and OpenClaw's **Model ID** field expect. The display to see.

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

Tool calls aren't executed : The model needs to support tool calling for client-side tools (`tools` / `tool_choice`). For Unsloth's built-in tools, remember to set `enable_tools: true` **and** list the ones you want in `enabled_tools` (e.g. `["python", "web_search"]`).

[<](#)

[Previous](#)
Qwen3.5 GGUF
Benchmarks

[Next](#)
Inference & Deployment

[>](#)

Last updated 6 days ago

Was this helpful?



Community

Reddit r/unsloth

Twitter (X)

LinkedIn

Resources

Tutorials

Docker

Hugging Face

Company

Unsloth Studio

Contact

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).

Events



© Unsloth, 2026

This site uses cookies to deliver its service and to analyze traffic. By browsing this site, you accept the [privacy policy](#).